a) Realice la elección de un diseño para esta realidad.

Debe indicar los Tipos Abstractos de Datos elegidos, por ejemplo:

Vehículos = … Secuencia (validado)

Vehículo = … Matricula X marca X año

Auto = … cantidad base X precio base x vehiculo

Moto = … cilindrada X precio base X vehiculo

También debe incluir observaciones que justifiquen la elección realizada.

Observaciones:

- Se elige Secuencia para la colección de vehículos porque permite agregar y eliminar elementos, y recorrerlos para calcular sumas, máximos, mínimos y conteos.
- Se modela Vehículo como clase abstracta porque no existe un vehículo genérico — siempre es un Auto o una Moto. El método `precioAlquilerDiario()` es polimórfico porque el cálculo es diferente según el tipo.
- Auto y Moto heredan de Vehículo, incorporando sus atributos específicos (cantPuertas y cilindrada respectivamente) más precioBase.

b) Realice el correspondiente desglose de requerimientos.

Para cada requerimiento, indique:

- Entrada
- Salida
- Proceso

Debe desglosar al menos los siguientes casos:

Requerimiento 1a: Alta de vehículo.

- Entrada: Datos del vehículo
- Salida: Vehículo ingresado
- Proceso:

    SI matricula ya existe, entonces

        Error: la matrícula ya existe

    Sino

        Crear objeto Auto o Moto con los datos ingresados

        Agregar vehículo a la secuencia

Requerimiento 1b: Baja de vehículo.

- Entrada: Matrícula del vehículo a eliminar
- Salida: Vehículo eliminado del sistema
- Proceso:

SI matrícula no existe, entonces

Error: la matrícula no existe

SINO

Eliminar vehículo de la secuencia

Requerimiento 2: Calcular suma de precios de alquiler diario.

- Entrada: Ninguna
- Salida: Valor numérico (double) con la suma total
- Proceso:

suma = 0

PARA CADA vehículo en la secuencia

suma = suma + vehículo.precioAlquilerDiario()

RETORNAR suma

Requerimiento 3: Obtener vehículos con mayor y menor precio de alquiler diario.

- Entrada: Ninguna
- Salida: Datos del vehículo con mayor precio y datos del vehículo con menor precio
- Proceso:

mayor = primer vehículo de la secuencia

menor = primer vehículo de la secuencia

PARA CADA vehículo en la secuencia

SI vehículo.precioAlquilerDiario() > mayor.precioAlquilerDiario()

mayor = vehículo

SI vehículo.precioAlquilerDiario() < menor.precioAlquilerDiario()

menor = vehículo

RETORNAR mayor.obtenerDatos() y menor.obtenerDatos()

Requerimiento 4: Contar motos por cilindrada.

- Entrada: cilindrada (int)
- Salida: Cantidad de motos con esa cilindrada (int)
- Proceso:

  contador = 0

  PARA CADA vehículo en la secuencia

    SI vehículo es instancia de Moto

      SI moto.getCilindrada() == cilindrada

        contador = contador + 1

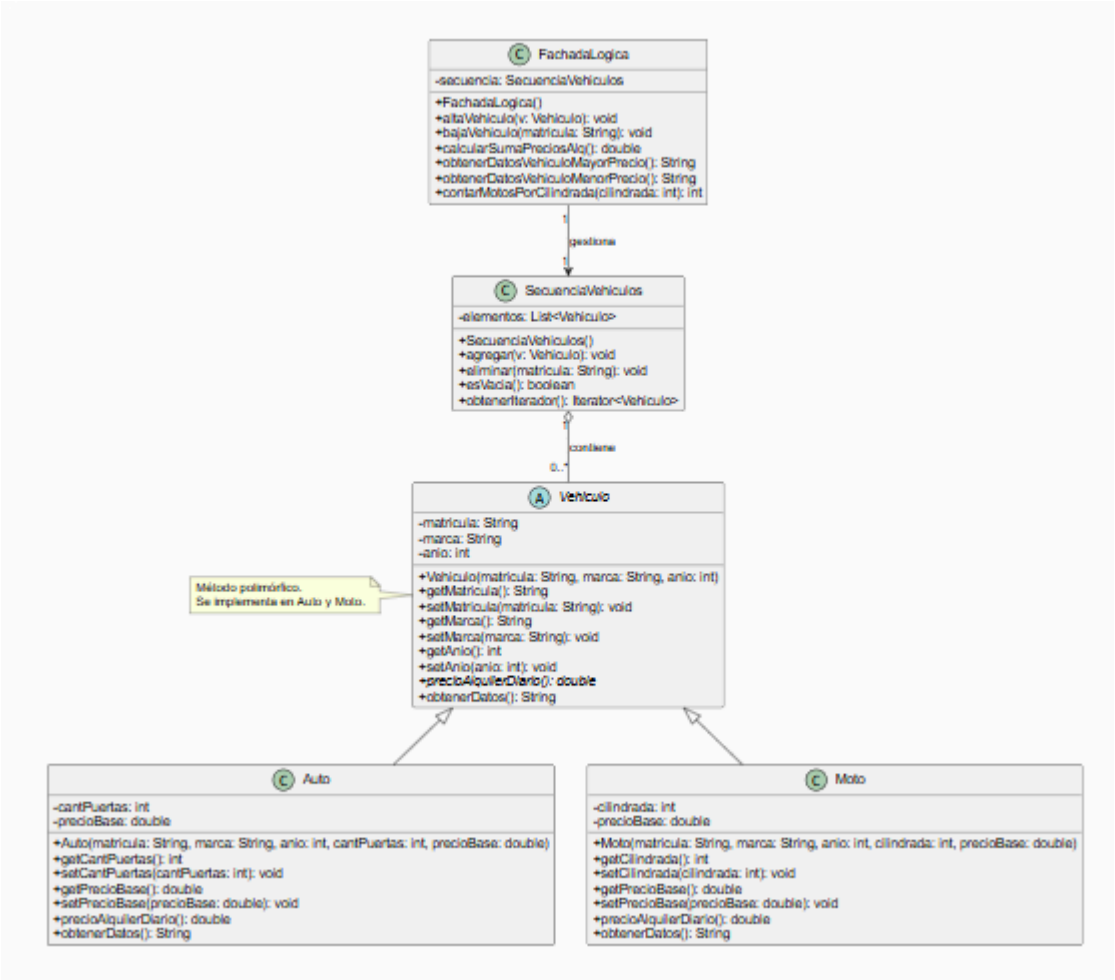
  RETORNAR contador

---

c) En base a las partes anteriores, realice la especificación del diseño elegido.

Debe presentar el **Diagrama de Clases de Implementación en UML**, mostrando solamente los atributos, y luego detallar la especificación de cada clase por separado, incluyendo:

- atributos;
- constructor;
- selectores;
- modificadores necesarios;
- métodos específicos;
- métodos polimórficos, si corresponde.




```
@startuml
skinparam classAttributeIconSize 0

abstract class Vehiculo {
  -matricula: String
  -marca: String
  -anio: int
  +Vehiculo(matricula: String, marca: String, anio: int)
  +getMatricula(): String
  +setMatricula(matricula: String): void
  +getMarca(): String
  +setMarca(marca: String): void
  +getAnio(): int
  +setAnio(anio: int): void
  +{abstract} precioAlquilerDiario(): double
  +obtenerDatos(): String
}

class Auto {
  -cantPuertas: int
  -precioBase: double
  +Auto(matricula: String, marca: String, anio: int, cantPuertas: int, precioBase: double)
```

```
  +getCantPuertas(): int
  +setCantPuertas(cantPuertas: int): void
  +getPrecioBase(): double
  +setPrecioBase(precioBase: double): void
  +precioAlquilerDiario(): double
  +obtenerDatos(): String
}

class Moto {
  -cilindrada: int
  -precioBase: double
  +Moto(matricula: String, marca: String, anio: int, cilindrada: int, precioBase: double)
  +getCilindrada(): int
  +setCilindrada(cilindrada: int): void
  +getPrecioBase(): double
  +setPrecioBase(precioBase: double): void
  +precioAlquilerDiario(): double
  +obtenerDatos(): String
}

class SecuenciaVehiculos {
  -elementos: List<Vehiculo>
  +SecuenciaVehiculos()
  +agregar(v: Vehiculo): void
  +eliminar(matricula: String): void
  +esVacia(): boolean
  +obtenerIterador(): Iterator<Vehiculo>
}

class FachadaLogica {
  -secuencia: SecuenciaVehiculos
  +FachadaLogica()
  +altaVehiculo(v: Vehiculo): void
  +bajaVehiculo(matricula: String): void
  +calcularSumaPreciosAlq(): double
  +obtenerDatosVehiculoMayorPrecio(): String
  +obtenerDatosVehiculoMenorPrecio(): String
  +contarMotosPorCilindrada(cilindrada: int): int
}

note left of Vehiculo
  Método polimórfico.
  Se implementa en Auto y Moto.
end note

Vehiculo <|-- Auto
Vehiculo <|-- Moto
FachadaLogica "1" --> "1" SecuenciaVehiculos : gestiona
```

SecuenciaVehiculos "1" o-- "0..*" Vehiculo : contiene

@enduml